

Watercolour rendering in a 3D real-time
environment using temporal coherence
techniques

Daniel Wooldridge

Bachelor of Science (BSc) in Computer Games
Application Development, 2025

School of Design and Informatics
Abertay University

Table of Contents

Table of Figures	4
Table of Tables	5
Acknowledgements	6
Abstract	7
Abbreviations, Symbols and Notation.....	8
Chapter 1 Introduction.....	9
1.1 Background	9
1.2 Research Question	10
1.3 Aim	10
1.4 Objectives.....	11
Chapter 2 Literature Review.....	12
2.1 Non-Photorealistic Rendering.....	12
2.1.1 Object Space and Image Space.....	12
2.1.2 Image space effects and convolution kernels.....	13
2.1.3 Edge detection	14
2.2 Painterly non-photorealistic rendering	16
2.2.1 Stroke-based rendering.....	16
2.2.2 Directed filtering techniques	17
2.3 Temporal coherence in NPR post-processing	19
2.4 Real-time Optimisation issues	20
Chapter 3 Methodology	22
3.1 Overview.....	22
3.1 Research and preparation.....	22
3.1.1 Technology used.....	22
3.1.2 Post-processing graphics pipeline architecture	22
3.1.3 Implementation design	23
3.1.4 Visual Inspiration.....	24

3.1.5 Stylisation Evaluation Criteria.....	25
3.2 Practical Implementation	27
3.2.1 Render Pass Breakdown	27
3.2.1.1 Depth Pass.....	27
3.2.1.2 Scene Rendering	27
3.2.1.3 RGB to Y'CbCr Conversion.....	27
3.2.1.4 Structure Tensor and Flow Map Generation	28
3.2.1.5 Bilateral Filtering (Edge).....	32
3.2.1.6 Difference of Gaussians	34
3.2.1.7 Flow-Guided Difference of Gaussians.....	37
3.2.1.8 Bilateral Filtering for Abstraction	38
3.2.1.9 Colour Quantisation, Blending and Paper texture	39
3.2.2.0 Temporal Blending	42
3.2.2.1 Colour Conversion and Comparison Shader.....	43
Chapter 4 Results.....	44
4.1 Stylisation output - Simulating watercolour effects.....	44
4.2 Temporal Coherence and Frame Stability	46
4.3 Performance and Real-Time Viability	47
Chapter 5 Discussion	49
5.1 Results Discussion	49
5.1.1 Visual Fidelity and Stylisation Effectiveness.....	49
5.1.2 Temporal Coherence and Frame Stability.....	52
5.1.3 Performance and Real-Time Viability	53
5.2 Overall Results Discussion	54
Chapter 6 Conclusion and Future Work	55
List of References	57
Bibliography	59
Appendices	60

Appendix A – Images.....	60
A.1 – Original Scene	60
A.2 – Colour Conversion RGB – YCBCR.....	60
A.3 – Structure Tensor	61
A.4 - Structure Tensor – Horizontally Smoothed	61
A.5 – Flow Map Generation	62
A.6 – Bilateral Filter - Vertical.....	62
A.8 – Difference of Gaussians.....	63
A.9 – Flow-Guided Difference of Gaussians	64
A.10 – Colour Quantisation	64
A.11 – Combining Shader	65
A.12 – Paper Overlay.....	65
A.13 – NPR watercolour outputs	66
A.14 – Texture Detail Comparison	66
A.15 - Temporal Incoherence.....	66
A.16 – GPU Usage.....	67
A.17 – Project Flow Chart	68

Table of Figures

Figure 1: Original Scene, including water, ship model and moving objects.....	60
Figure 2: RGB to YCBCR conversion.....	60
Figure 3: Structure Tensor	61
Figure 4: Horizontally smoothed Structure Tensor	61
Figure 5: Vertical smoothed Structure Tensor and Flow map Generation.....	62
Figure 6: Vertical Bilateral Filter	62
Figure 7: Horizontally smoothed Bilateral Filter.....	63
Figure 8: Difference of Gaussians	63
Figure 9: Flow-Aligned Difference of Gaussians	64
Figure 10: CQ - Colour Quantisation	64
Figure 11: Combining CQ and Edge Abstraction	65
Figure 12: Paper Texture Overlay	65
Figure 13: Comparison with Curtis and Bousseau result: (a) original image, (b) Curtis watercolour result, (c) Bousseau watercolour result, (d) my result.....	66
Figure 14: Importance of Texture Detail	66
Figure 15: Stylisation inconsistencies across frames	66
Figure 16: GPU Usage of each Render Pass.....	67
Figure 17: Pipeline Flow chart.....	68

Table of Tables

Table 1: Evaluation Qualitative Matrix	25
Table 2: Evaluated NPR Techniques, using Evaluation Qualitative Matrix.....	45
Table 3: Render Pass GPU Times (ms)	47

Acknowledgements

I would like to thank my supervisor, Erin Hughes for the amazing advice and guidance throughout this project. Without her support, I never would have created something I have been wanting to try for a long time.

Thank you to Ruth Falconer for subbing in for a few weeks when Erin couldn't make it. Having someone look and give encouragement makes a massive difference.

Thank you to my family and my girlfriend for having my back, especially with the last month being cruel and unrelenting.

Lastly, to all the friends who have helped me during this stressful time, from the deep conversations to the unrelenting laughter, I appreciate you for keeping me grounded at times like these.

Abstract

Non-photorealistic rendering (NPR) techniques offer artistic and stylised visual outputs, such as painterly effects and specifically watercolour via image-space calculations. However, many challenges can occur when doing so. These challenges range from temporal consistency and capturing the visuals that attempt to recreate traditional art styles, from brushstrokes to texturing. Without temporal consistency, flickers, jitters and random visual artefacts may occur, ruining player immersion when in a game environment. This project aimed to develop a watercolour post-processing effect that has little temporal inconsistency by achieving smooth transitions between frames, whilst replicating the unique features of watercolour such as pigment diffusion, soft edges, and the way water interacts with canvases, whilst also maintaining computational efficiency so that it can run a game scene within standard rendering time. The development process of this project followed that of Kyprianidis and Döllner's (2009) approach to create such an effect whilst attempting to add temporal coherence techniques. The project was then evaluated using a qualitative matrix against other NPR watercolour effects that have come previously, highlighting visual details that enhance or diminish the watercolour aesthetic. Followed by this, screenshots were taken and broken down, looking at texture detail differences for the post-process effect and how this influences the output. More screenshots were taken again to view the visual consistency over time, to view and acknowledge how well the project's output is visually consistent. Finally, each render pass's rendering time was measured and viewed. This project was deemed a mid-success in the creation of watercolour effects whilst also keeping unwanted visual artefacts at a minimum, whilst excelling in GPU rendering times.

Abbreviations, Symbols and Notation

GPU – Graphics Processing Unit

NPR – Non-photorealistic Rendering

SBR – Stroke-based Rendering

GUI – Graphical User Interface

DoG – Difference of Gaussians

FDoG – Flow-aligned Difference of Gaussians

CPU – Central Processing Unit

TC – Temporal Coherence

Chapter 1 Introduction

1.1 Background

Non-photorealistic rendering (NPR) has allowed for artistic stylistic choices in computer graphics since 1994 which was coined by (George W & David H. Salesin, 1994) beyond attempting to achieve realistic image surface representation. NPR leverages mathematical formulas to modify pixels and vertices on a computer screen to replicate traditional art styles and techniques, including ink, pencil sketching, oil and watercolour paintings. Initial image effects manipulated 2D static images but post-processing in real-time 3D rendering allows for NPR in video, computer games and other real-time applications.

However, real-time applications can illuminate temporal inconsistencies, especially in techniques that are estimated numerically. Unwanted noise and visual incoherency are two main issues with applying NPR filters for stylisation. This is caused by how the NPR filters are applied, frame-by-frame complex pixel manipulation occurs on the render textures that have been passed through to the shader. As this happens major changes can occur for example a player moves or a cinematic is triggered, this difference of pixel values per frame can lead to the creation of visual artefacts or irregularities, this is called temporal incoherence (TC). TC stops immersion and breaks believability which negatively impacts the user experience creating it a space for research as it destroys creative integrity.

Post-processing and shader-based techniques can be used within a modern graphics pipeline to create stylized outputs of a game scene without needing to modify the underlying geometry. Traditional watercolour offers a rich visual experience based on edge behaviour, tone separation, bleeding and diffusion. All of which can be approximated through directed filtering and stylized image operations in image space. The project focuses on simulating these characteristics using flow maps,

structure tensors and a series of post-processing passes that can replicate watercolour-like features.

Many commercial games have used NPR for expressive storytelling, visual identity, or aesthetic differentiation. Games such as *Okami* (Clover Studio, 2006) and *Return of the Obra Dinn* (Pope, 2018) explore stylization to different degrees, influencing player perception and narrative tone. This project explores watercolour simulation through the lens of post-processing to examine how such techniques can be realized in real time with visual fidelity and stability.

1.2 Research Question

How can flow maps, structure tensors, and other-directed filtering techniques be used to simulate watercolour bleeding effects in a post-processing pipeline, while ensuring temporal coherence across games in real-time within a 3D game environment?

1.3 Aim

To develop a watercolour post-processing effect that simulates natural colour bleeding and pigment diffusion in traditional watercolour paintings whilst employing temporal coherence techniques to maximise visual consistency and optimising computational performance for real-time 3D game environments.

1.4 Objectives

The project's objectives are:

- Simulate watercolour effects in a post-processing pipeline, focusing on replicating natural pigment diffusion, soft edge behaviour, and how paper texture affects the fluidity of the paint observed in traditional watercolour paintings.
- Implement temporal coherence techniques to reduce flickering, jittering, and instability caused by per-frame stylisation changes in real-time scenes.
- Optimise computational performance to ensure the effect is viable in real-time 3D applications without introducing significant frame drops or latency.

The thesis will examine the literature surrounding non-photorealistic rendering (NPR), stylization techniques, and edge-aware image processing, providing theoretical background necessary to contextualise the project. It will define the system requirements and describe the implementation of a custom rendering pipeline and associated shaders to meeting the aims and objectives outlined above. The research will also report and discuss the findings derived from visual analysis and performance testing of the developed artifact, as structured in the methodology. Finally, conclusions will be drawn based on the effectiveness and limitations of the approach and recommendations will be provided for future work – highlighting potential improvements, alternate methods, broader applications in areas such as real-time graphics and animation.

Chapter 2 Literature Review

2.1 Non-Photorealistic Rendering

2.1.1 Object Space and Image Space

NPR techniques can be broadly categorised into Object Space and Image Space methods (Bénard et al., 2011).

Object space techniques function on 3D geometry, ensuring the stylisation is anchored to the object's structure. These methods are highly successful in maintaining details across many frames but can be computationally expensive as it increases processing load which can make them less suitable for real-time rendering contexts such as games.

Image space techniques process rendered 2D images and are widely used in post-processing effects due to their efficiency and independence from scene geometry, such as painterly rendering. Painterly rendering is a method of simulating hand-crafted painting styles such as oil, ink (Clover Studio, 2006), or watercolour using algorithmic image filters. These effects replicate the expressive and interpretive qualities of traditional painting, prioritising stylisation and abstraction over photorealism. As a post-process, it allows for real-time implementation without modifying scene geometry, which is essential for NPR pipelines (Kyprianidis et al., 2009).

A significant downside of image-space NPR is temporal incoherence. This occurs as each frame is processed independently, creating visual artefacts such as flickering, jittering, or any instability, especially in dynamic scenes. This has many issues, such as player immersion can be hindered due to a lack of consistency in the post-processing effect. Methods to solve these temporal issues include optical flow techniques (Vanderhaeghe et al., 2007) and frame blending (Bénard et al., 2011), with both techniques aiming to smooth out frame-to-frame transitions.

However, these methods also come with some downsides, however, such as computational overhead or it may introduce other visual artefacts.

2.1.2 Image space effects and convolution kernels

Image space effects are a foundation in real-time graphics and stylisation because they operate directly on 2D rendered images, using calculations on the pixels to manipulate colour, contrast and structure. These operations commonly reference image coordinates, allowing filters to be applied across an image without needing knowledge of the scene's geometry. A highly used technique in image-space processing is convolution kernels, these perform basic mathematical operations over local pixel neighbourhoods. A convolution kernel is a small matrix, commonly 3x3 or 9x9, that moves along an image, calculating weighted sums based on a pixel's adjacent values. This allows for efficient edge detection, blurring, and is the foundation for a lot of stylisation effects as it modifies the pixel's intensity in a structured way. These operations are also effective as they take advantage of the hardware architecture of a GPU. Originally designed for vector and matrix-based computations, GPUS are highly optimised for multiply-add (mul-add) instructions, the core operation in convolution. Furthermore, because convolution only requires nearby pixels, rather than the full image, this allows for a high degree of parallelism. Thousands of these local operations can be executed simultaneously, making convolution kernels both computationally efficient and ideal for real-time graphics pipelines. Convolution kernels are a foundation for many image-space processing techniques in computer graphics, such as famous techniques like Gaussian blur and Sobel filtering Kin and Casper,

2013). These methods operate by applying a weighted matrix to a pixel and its neighbours, modifying each pixel's value based on the values around it. For example, Gaussian blur applies a weighted matrix with smooth falloff values to produce a soft blur effect, while Sobel filtering creates edges by detecting gradient changes across neighbouring pixels. More complex stylisation operations, such as Difference of Gaussians (DoG), are direct extensions of these basic techniques. For example, DoG applies two Gaussian blurs with different radii and subtracts them to enhance mid-frequency details (Winnemöller et al., 2012), creating edge abstraction. Similarly, bloom effects and edge-aware filters rely on basic filtering operations.

2.1.3 Edge detection

Edge detection is critical to achieving the project's goal of replicating painterly watercolour aesthetics in real-time post-processing. As outlined in the aims, replicating soft edge behaviour is a central goal, and edge detection techniques provide the foundational data from which expressive contours and stylised abstractions are derived. Several methods exist for implementing edge detection, but among the widely used in NPR convolution-based techniques are Sobel filtering, the difference of Gaussians and Laplacian-based methods (Kyprianidis et al., 2009). These methods analyse and use the 3x3 matrix from the convolution kernel to estimate intensity gradients, effectively distinguishing edges from smooth regions.

Despite the effectiveness of techniques such as these, edge detection techniques in NPR are also prone to the issue of temporal inconsistency. This happens because when mathematically producing these edges using 3x3 kernels with dynamic scenes and

camera motion, the edges can be calculated differently and staying consistent with previous calculations can lead to flickers and unwanted visual artefacts. To mitigate this, temporal blending (Bénard et al., 2011) and optical flow-guided filtering (Vanderhaeghe et al., 2007) have been proposed, these methods stabilise edges across frames by tracking motion vectors and averaging contours over time.

Additionally, flow-guided filtering techniques, such as flow-aligned Difference of Gaussians (FDOG), have been proposed. These help create more stylised edges but also help with maintaining coherence. This method builds on traditional DoG filtering by introducing direction, coordinating the convolution process with dominant image gradients. Kyprianidis and Döllner (2009) utilise a structure tensor to estimate these directions, allowing the filter to follow the natural flow of the image. This approach enhances edge abstraction while preserving structural coherence, which is essential in NPR. Although primarily developed for static images, the theoretical basis of FDoG has influenced subsequent research into temporally stable stylisation for animation and games.

2.2 Painterly non-photorealistic rendering

2.2.1 Stroke-based rendering

Stroke-based rendering (SBR) simulates traditional paintings or drawing techniques by applying techniques that imitate individual brush strokes to represent the scene. A method proposed by Möller et al. (2018) creates an image using brushstrokes that mimic the flow and texture of traditional art, such as oil, watercolour painting, or ink sketches. Techniques like these offer a high level of stylisation and control by enabling manipulation of stroke attributes, for example, orientation and thickness, these strokes can also be aligned to scene geometry procedurally. This process is done by identifying key features in the image such as edges or high-contrast areas and then applies strokes that follow the natural structure of the scene or image by aligning them with local image gradients. Kuwahara filtering (Kyprianidis et al., 2009) and flow-based Difference of Gaussians (Kyprianidis and Döllner, 2009). These methods ensure that strokes align with the underlying geometry of the image, improving visual consistency across frames.

Whilst SBR is effective for producing stylised NPR visuals, it can have some computational overhead, particularly when applied to real-time applications such as games. The complexity of managing thousands of dynamic strokes, while also preserving edge details and coherence, places significant demands on the GPU. These include memory usage, shader throughput and arithmetic logic unit operations shown by Ciao et al. (2024). To mitigate these challenges, modern approaches use combinations of filtering techniques and linear algebra tensors to define the surface attributes and reduce noise. Specifically, bilateral filtering, structure tensors presented in Kyprianidis et al. (2009), Kyprianidis and

Döllner (2009) provide edge-aware smoothing and orientation information, allowing strokes to be distributed consistently along the natural flow of the image.

Further advancements in GPU-accelerated rendering have further developed SBR techniques, especially in real-time applications. Ciao et al. (2024). present a framework that exploits modern GPU capabilities, more specifically the use of geometry and compute shaders, that are used for vector brush stroke rendering. These shaders are used for instanced rendering and stroke placement using prefix-sum calculations. Strokes are represented as paths and use footprint-based sampling to form realistic deformation and spacing. This creates consistency within the brush strokes, which creates temporal coherence in real-time.

2.2.2 Directed filtering techniques

To enhance abstraction and stylisation of images, but to also maintain significant structural details consistently, directed filtering methods use the structural components in the scene for processing outcomes. Increasing features such as edges, contours and textures are preserved and emphasised in a controlled manner. The Kyprianidis and Döllner (2009) approach implements flow-based Difference of Gaussian filtering, which utilises streamlines derived from a structure tensor to direct the filtering process. Unlike standard convolution-based edge detection methods which apply uniform kernels across the full image, this technique aligns filtering along dominant image structures, ensuring edges are reinforced without disrupting the natural flow of the scene.

By using structure tensors, this method adapts filtering based on local image gradients, allowing for an edge-preserving smoothing effect that still maintains high detail.

However, an issue that Kyprianidis and Döllner's (2009) implementation and NPR in general face is that it is used for images and not for video animation or real-time use, such as in games. Since the method processes each frame separately, inconsistencies can emerge between frames that are in rapid succession, leading to noticeable flickering and instability. These temporally incoherent artefacts arise because the process lacks motion-tracking mechanisms, either for objects within the scene or for the movement of the camera itself. Kyprianidis and Döllner's (2009) method provides strong structural coherence in still images and less dynamic scenes; however, it is insufficient in dynamic game scenes as they tend to include intense motion.

2.3 Temporal coherence in NPR post-processing

In NPR, temporal coherence refers to the consistency of stylised effects across consecutive frames in animations or real-time applications. Unlike photorealistic rendering, where lighting, shading, and geometry transforms naturally provide smooth frame-to-frame transitions, NPR techniques often rely on image-space filtering methods that are applied to each frame independently (Bénard et al., 2011). Common side effects of this are unwanted flickering, visual artefacts and instability, which breaks immersion in real-time applications such as video games. The challenge lies in ensuring that stylised effects such as SBR, texture manipulation and edge detection remain visually coherent and consistent across motion and transformations whilst preserving the wanted stylised approach.

Several approaches have been proposed to mitigate these issues. Frame blending techniques attempt to smooth transitions by blending pixel values across multiple frames, reducing flickering at the cost of ghosting effects (Bénard et al., 2011). Ghosting is the aftermath of blending as the frames create a blurry, trailing effect behind moving objects because of the averaging of pixel values. Optical flow-based methods track the motion of pixels between frames, aligning stylised elements with the underlying scene movement to improve stability (Vanderhaeghe et al., 2007). Using techniques like these brings more problems into the equation, such as computational overhead, making them less feasible for game environments.

Despite these methods and the advancements, achieving real-time, temporally coherent NPR effects remains a complex challenge. Techniques must balance stability, stylistic needs, and performance

which requires complex solutions that use previous frame data whilst maintaining responsiveness in dynamic scenes.

2.4 Real-time Optimisation issues

NPR techniques often come with significant computational overhead, making real-time applications challenging. Many approaches rely on per-pixel operations or iterative algorithms, which scale poorly with larger resolutions and scene complexity. Kyprianidis et al. (2009) highlight that anisotropic Kuwahara filtering, while effective for edge preservation and abstraction, is computationally expensive and intense, especially in real-time video processing. Additionally, Bénard et al. (2011) discuss how methods designed to improve temporal coherence, such as frame blending and optical flow-based tracking, often increase the computational costs, making it impractical for real-time use in high-performance environments when solving the issue itself. This is because the program will now need to store multiple frames and processes simultaneously, which increases memory usage and bandwidth requirements. Furthermore, when using motion data, the program computes motion vectors for every pixel between consecutive frames.

Kumar et al. (2019) further conclude that achieving a balance between computational efficiency and visual quality remains a large challenge in NPR. The paper survey highlights that while modern advances in GPU acceleration and parallel computing have improved performance, many state-of-the-art NPR techniques still struggle to meet the demands of real-time rendering. Techniques such as GPU-accelerated vector brushstrokes (Ciao et al., 2024) represent a step forward in improving efficiency, yet they still require

significant processing power to maintain high-quality stylisation at interactive frame rates.

Optimisation in NPR is a major issue: whilst stylising processing is computationally heavy, so are the other factors necessary for a successful real-time environment (e.g. temporal coherence)

Chapter 3 Methodology

3.1 Overview

This chapter outlines the methodology for developing a real-time post-processing pipeline capable of simulating watercolour stylisation within a 3D environment. The approach specifically explores how flow maps, structure tensors, and directed filtering techniques can be used to replicate the visual characteristics of watercolour painting, such as colour bleed and canvas interaction, purely through screen space operations.

This project also investigates the challenges of temporal coherence within a game environment, with a moving camera and object movement.

3.1 Research and preparation

Before implementation could begin, it was essential to research both the visual characteristics of traditional watercolour and the feasibility of reproducing these effects in a real-time, post-processed pipeline. Examining previous watercolour NPR effects and how shaders could be used to create such an effect.

3.1.1 Technology used

The technology chosen for this project is as follows:

- C++
- DirectX11
- HLSL (high-level shader language)

With the addition of DXTK, DirectX Tool Kit as it simplifies basic tasks.

3.1.2 Post-processing graphics pipeline architecture

The core of this project is built around a custom post-processing graphics pipeline designed to simulate watercolour stylisation. This pipeline

consists of a series of sequential render passes that operate on render targets of the full window.

Each frame undergoes multiple stages of the post-process, with render textures used to store the output of each step. These render textures are later used in the pipeline, creating layered effects. The pipeline follows a modular structure, making it easier to debug, scale, and isolate specific stylisation techniques so that the desired output can be easily modified for a user-specific output. Follow Figure 17: Pipeline Flow chart, to see a diagram showing the pipeline.

3.1.3 Implementation design

The implementation follows Kyprianidis and Döllner's (2009) method to create a foundation that the watercolour post-processing effect can be built on top of, whilst maintaining real-time performance and temporal coherence. To achieve this, the pipeline was designed using a series of clearly defined full-screen passes, using render targets and feeding them into each subsequent stage.

Each shader in the pipeline was written to be self-contained, enabling easier debugging and adjustment without compromising the rest of the program. This structure allowed the project to remain highly readable—individual effects could be isolated, evaluated and changed independently.

All shaders used throughout the project are instantiated early in the application using an `InitiliasShader()` method. Control parameters for all filters and stylisation techniques are defined in `InitialiseVariables()`. These include the settings for every filter that takes in parameters:

- Bilateral filtering: spatial and range sigmas
- Difference of Gaussian: edge sensitivity, smoothing spread, and contrast
- Flow curve traversal: step size, start position, and tangent direction
- Paper overlay blending: depth influence and texture strength
- Temporal coherence: blend strength and previous frame texture

- Colour quantisation: number of tone bins and transition smoothing

All these parameters can be changed during runtime via the GUI, ImGui. Creating the pipeline like this allows us to keep the code readable and scalable.

3.1.4 Visual Inspiration

To understand the visual goals of this project, many real-world and digital examples of watercolour art were studied. These examples helped to define the aesthetic goals of the post-processing pipeline, focusing on texture, pigment behaviour and edge treatment.

Unlike photorealistic rendering or other painterly effects, watercolour painting relies heavily on fluidity, like pigment bleeding, soft edge transitions, and how paint interacts with the canvas texture. These properties informed many of the technical decisions made in the implementation, such as following Kyprianidis and Döllner's (2009) method, as this includes techniques such as flow-aligned filtering, edge abstraction, and adding to this implementation with paper texture and such.

The study involved analysing previous attempts at creating watercolour through the means of NPR, their work demonstrates as follows:

- Loose, expressive edges rather than hard outlines
- Layered tonal washes with clear quantised value bands
- Flow-driven pigment diffusion that follows surface contours
- Paper grain influencing colour accumulation and drying behaviour

Screenshots and physical references were collected throughout development and used to evaluate how well the output aligned with the watercolour aesthetic.

3.1.5 Stylisation Evaluation Criteria

To assess the success of the final stylised output, a qualitative evaluation framework was developed based on observed characteristics of traditional watercolour paintings. This framework, a qualitative matrix, defines several visual traits considered central to the watercolour aesthetic. These traits were identified through visual analysis of traditional watercolour artwork and stylized rendering examples. They reflect the aesthetic goals of the project and informed the post-processing design.

Trait	Description	Evaluation Criteria
Edge Softness	Degree to which edges appear soft or “bleed” naturally, recreating pigment dispersion	Sharp / Slightly Soft / Soft / Bleeding
Tonal Transitions	Presence of distinct tonal regions, recreating dried pigment	None / Subtle / Moderate / Very
Flow Alignment	Whether pigment or tonal transitions follow natural contours or appear directional	None / Partial / Good / Strong
Paper Influence	Visual influence of paper grain on pigment accumulation	Not visible / Mild / Clear / Strong
Pigment Diffusion	How well the transition between tones mimics real pigment spreading	Hard edges / Smooth / Organic / Water-like
Edge Abstraction	Stylisation of edges to resemble brush lines rather than geometric edges	Sharp / Slightly stylised / Stylised / Expressive

Table 1: Evaluation Qualitative Matrix

To make sure grading was kept consistent for each trait shown in the matrix, options were given to select from. Edge softness was graded from "Sharp" to "Bleeding", Tonal transitions from "None" to "Very", Flow alignment from "None" to "Strong", Paper influence from "Not visible" to

"Strong", Pigment diffusion from "Hard edges" to "Water-like" and finally Edge abstraction going from "Sharp" to "Expressive".

While the aims of this project only focused on a few of these, other watercolour traits have been included to evaluate the full range of traits within watercolour. Using this, the project will be able to evaluate the outcome consistently and scientifically.

3.2 Practical Implementation

Showing the implementation for the project one pass at a time to understand the different NPR and standard shader techniques that were used to create the watercolour post-processing effect.

3.2.1 Render Pass Breakdown

3.2.1.1 Depth Pass

The depth pass captures the scenes depth buffer into a separate render target. This is used later for depth-aware blending, particularly when applying paper texture and for modulating stylisation intensity based on object distance. It ensures that effects like the canvas overlay respond natural to the 3D structure of the scene.

3.2.1.2 Scene Rendering

This pass renders the whole scene, from models to textures. This serves as the base image that the post-processing pipeline will stylise. To see the original scene, Figure 1: Original Scene, including water, ship model and moving objects.

3.2.1.3 RGB to Y'CbCr Conversion

To stop certain visual artefacts that can keep the simulation from not being temporally coherent, we can convert into Y'CbCr colour space. Y'CbCr colour space stands for the following:

- Y: Luminance
- Cb: Blue chrominance
- Cr: Red chrominance

Converting from RGB into the Luminance and Chrominance colour space enables stylisation techniques such as edge detection and bilateral filtering to operate more effectively on brightness information whilst preserving colour fidelity.

To convert from one to the other, a transformation can be used defined by the following matrix multiplication:

$$\begin{pmatrix} Y \\ C_b \\ C_r \end{pmatrix} = \begin{pmatrix} 0.299 & 0.587 & 0.114 \\ -0.169 & -0.331 & 0.500 \\ 0.500 & -0.419 & -0.081 \end{pmatrix} \begin{pmatrix} R \\ G \\ B \end{pmatrix}$$

In addition to this, a value of 0.5 is added to the chrominance values to prevent negative values.

Once in Y'CbCr, most of the stylisation pipeline operates on the Y' channel. This enables consistent and meaningful manipulation of contrast, tone, and edges without introducing unwanted colour shifts or instability between frames. The result is converted back into RGB for output.

The visuals when converting into the Y'CbCr colour space can be found here - Figure 2: RGB to YCBCR conversion

3.2.1.4 Structure Tensor and Flow Map Generation

To create a stable foundation for the project, a structure tensor was created following the Kyprianidis and Döllner (2009) method.

Implementing a structure tensor begins with identifying the local intensity changes across the image, which is a process known as gradient estimation. To do this, a Sobel operator is used, a convolution kernel filter that is commonly used for edge detection. It works by estimating the gradient of an image in both directions, effectively highlighting areas of high change by using a simple matrix that gives more weight to pixels closer to the centre pixel.

To compute these gradients, denoted as l_x and l_y , the Sobel filter is applied using predefined weighting values. These values from the 3x3 convolution kernel that is used on the render texture highlight edges and general regions that have high value changes. The matrix can be defined as:

$$G_x \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix} \quad G_y \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$$

These gradients are then used to compute the elements of the structure tensor matrix, specifically $I_x^2, I_x I_y, I_y^2$ – Which are encoded as red, green and blue channels of a render texture. These values are then used to understand the orientation and flow of the image.

To keep things efficient, the shader stores the x and y gradients as vectors, u and v, which hold the I_x and I_y , respectively. The tensor components are then calculated using the dot product as follows:

- $E = u \cdot u = I_x^2$
- $F = u \cdot v = I_x I_y$
- $G = v \cdot v = I_y^2$

The dot product $a \cdot b$ of two vectors $a = (a_1, a_2, a_3)$ and $b = (b_1, b_2, b_3)$ is defined as:

$$\bar{a} \cdot b = a_1 b_1 + a_2 b_2 + a_3 b_3$$

The dot product creates the tensor by combining the gradient values, I_x and I_y , into single values, which are then stored in the red, blue and green channels of the render texture, represented by E, F, and G.

After calculating the tensor components E, F, and G, they are smoothed in both X and Y directions using a Gaussian blur. This step is essential because the raw gradients can be noisy, especially in regions with high-frequency detail or subtle tonal transitions. The figures Figure 3: Structure Tensor and Figure 4: Horizontally smoothed Structure Tensor show the original structure tensor created and the blurred version.

Without smoothing, the direction information derived from these tensors would be unstable, resulting in inconsistent or inaccurate flow fields, particularly detrimental to stylisation techniques that rely on coherence orientation across an image.

To reduce computational costs, we do both blurs in different render passes, first the horizontal blur that uses the original structure tensor render texture and then the vertical pass that uses the horizontal render texture. Each pass applies predefined Gaussian weights to the adjacent texels around the current pixel. These weights are highest at the centre and taper off symmetrically, which creates nearby texels to have more influence over the distant ones.

The Gaussian weights used in this implementation follow a normalised 5-tap 1D kernel:

$$\{0.0625, 0.25, 0.375, 0.00625\}$$

From this structure tensor, a flow map can be created. A flow map shows how textures and the overall scene flows.

To create one the project computes the dominant eigenvector of the 2x2 symmetric structure tensor matrix at each pixel, representing the principal direction of intensity variation in the image.

The structure tensor is defined as:

$$T = \begin{pmatrix} E & F & F & G \end{pmatrix}$$

Where:

- $E = I_x^2$ - squared horizontal gradient
- $F = I_x I_y$ - cross product of the gradients
- $G = I_y^2$ - squared vertical gradient

The flow direction is then calculated using the eigenvectors from this matrix. Since the tensor is symmetric and positive semi-definite, the eigenvalues are always real, and its eigenvectors are always orthogonal. The eigenvector with the largest eigenvalue shows the direction of the greatest difference in the image, this is then used as the tangent direction in the flow map.

The dominant eigenvalue is calculated as:

$$\lambda_1 = \frac{E + G + \sqrt{(E - G)^2 + 4F^2}}{2}$$

And the associated eigenvalue can be derived using:

$$v = \left(\frac{F}{\lambda_1 - E} \right)$$

In this implementation, following Kyprianidis and Döllner (2009), the eigenvector is normalised and stored in a float 2 in the red and green channels. The flow map is an essential tool that is used throughout the pipeline, making it vital for this project.

However, a discovery through implementing this flow map into the project was that the flow map, if a value was close enough to 0 or out of bounds, would not store the eigenvector in float2 resembling colours. It would create a black, unsuccessful part of the render image, which causes many issues in further shaders. To fix this, a fallback value was added, however, this has made some sections of a texture fail to store not produce an effective watercolour aesthetic.

The final output can be seen in Figure 5: Vertical smoothed Structure Tensor and Flow map Generation.

3.2.1.5 Bilateral Filtering (Edge)

Following on from the flow map creation and following the implementation from Kyprianidis and Döllner (2009). A bilateral filter is used to smooth image content while preserving edge structures. The bilateral filter is a non-linear edge-preserving smoothing operator that applies a weighted average based on both proximity and similar intensities. Unlike a standard Gaussian blur, which treats all pixels in the kernel equally for intensity, the bilateral filter penalises differences in pixel values, which reduces blurring across strong edges.

This pass is repeated multiple times during the render pipeline, 1 before edge detection and 3 after serving for strengthening abstraction.

The bilateral filter equation used is:

$$f = \frac{\sum_{x \in \Omega(x_0)} f(x) G_{\sigma_d}(\|x - x_0\|) G_{\sigma_r}(\|f(x) - f(x_0)\|)}{\sum_{x \in \Omega(x_0)} G_{\sigma_d}(\|x - x_0\|) G_{\sigma_r}(\|f(x) - f(x_0)\|)}$$

Where:

- x_0 = centre of the filter window
- $\Omega(x_0)$ = local neighbourhood
- G_{σ_d} = spatial kernel, controlling proximity weighting
- G_{σ_r} = range kernel, weighting based on colour similarity

The Gaussian kernel used for both weighting functions is defined as:

$$G_{\sigma}(t) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{t^2}{2\sigma^2}\right)$$

These calculations are adapted to work efficiently in the render pipeline by aligning the filtering direction with the local structure of the image, which is taken from the flow map that was generated previously in the last render pass. The project performs two passes of filtering, the first along the gradient's direction, which suppresses noise across high-contrast

edges, and the second along the tangent's direction, reinforcing the flow of structure-aligned detail.

To identify which direction the shader should work on, a variable called “passing” is passed along.

The direction vector is computed as follows:

```
float2 direction = (passing == 0)
    ? float2(tangentFlow.y, -tangentFlow.x) // Gradient direction (perpendicular to flow)
    : tangentFlow;                          // Tangent direction (aligned with flow)
```

The project also applies a step size calculation to ensure that sampling remains uniform and avoids visual artefacts. Because the filter operates along non-axis-aligned directions, an improper implementation with a fixed step would result in non-uniform spacing of samples.

To solve this, the step length calculates how far to step on the render texture when travelling along a direction vector, ensuring correct spacing as it goes.

The step length s is computed as:

$$s = \frac{1}{\sqrt{1 + \left(\frac{dy}{dx}\right)^2}} \text{ if } |dx| > |dy|$$
$$s = \frac{1}{\sqrt{1 + \left(\frac{dx}{dy}\right)^2}} \text{ otherwise}$$

This equation effectively scales the movement along the direction vector so that you advance one texel in either the x or y direction per step, depending on which component is dominant, preventing visual artefacts.

When implementing the step length calculation instead of directly using the direction vector, the size of the step is calculated using the dominant component. A variable called “stepLength” acts as the dominant

component, and when multiplying this float value by the direction, it results in a one-textel shift along the image-screen axis.

Whilst building the bilateral filter in the project, the range attribute of the bilateral filter was viewed to create a colour bleed effect, helping towards one of the project's main goals, even if it is subtle. This is because of how the bilateral filter weighs colour differences during the calculation. On top of this, by applying the bilateral filter, the watercolour effect of having visible edges whilst having blended regions of colour stands prominently. This stage of the watercolour post-processing effect acts as a foundation for key traits that the artistic style holds. With blocky but correct flow for colour bleed via the range attribute and edge detection to reflect dry paint.

3.2.1.6 Difference of Gaussians

Once the bilateral filter has been completed, meaning directional smoothing has occurred, the next step in the post-processing pipeline is a Difference of Gaussian pass, a common edge detection technique used to extract edge information from the bilateral filter render texture which will be used to enhance our watercolour dry edges.

The DoG operates by subtracting two differently blurred versions of the same render target, typically using Gaussian filters with varying standard deviations. This results in a band-pass filter that highlights regions of high-frequency, i.e. edges, while suppressing low-frequency variations like smooth shading or lighting gradients.

This method approximates the Laplacian-of-Gaussian (LoG) operator, but is more efficient and separable, making it well-suited for real-time post-processing. In this project, the DoG pass is applied after the bilateral filters to enhance the abstraction of important visual contours. It will be the next part of the pipeline that focuses on simulating watercolour dry edges.

The DoG operation is shown as below:

$$\sum_{x \in \Omega(x_0)} G_{\sigma_e}(\|x - x_0\|) - \tau \cdot G_{\sigma_{e'}}(\|x - x_0\|)$$

Where:

- G_{σ_e} = Gaussian blurred version of the image
- $G_{\sigma_{e'}}$ = Gaussian blurred image with a wider spread $\sigma_{e'}$ ($\sigma_{e'} > \sigma_e$), capturing lower-frequency details
- τ = contrast adjustment factor

The result of this is a single edge strength value for each pixel.

In real-time applications, however, applying two full Gaussian filters per pixel is computationally expensive. To overcome this, the method proposed by Kyprianidis and Döllner (2009) performs the DoG operation within a single pass using a loop that accumulates weighted samples with two Gaussian distributions, one narrow and one wide, applied simultaneously (Marr and Hildreth, 1980). This is significantly more efficient and reduces memory bandwidth usage by eliminating the need for intermediate render targets.

The mathematical approach that calculates the filtered response uses the following expressions:

- $\exp(-\frac{d^2}{2\sigma_e^2})$ = the narrow Gaussian, capturing fine details
- $\exp(-\frac{d^2}{2\sigma_{e'}^2})$ = the wide Gaussian, preserving broader structure

These expressions compute the respective weights used to accumulate blurred values for the narrow and wide Gaussians, which are later subtracted and adjusted to form the final edge response.

This pipeline does both DoG filters in one shader pass, this is to keep computational costs low as it calculates the narrow and wide Gaussian weights at each step within the loop instead of through two loops and doing one at a time. This helps reduce texture reading and memory

usage low as it stops the need for having more render textures than necessary.

The sampling direction is normalised using the dominant axis of the normal vector, ensuring uniform texel steps regardless of gradient steepness, a technique like the one used before in the bilateral filter. At each step, the shader accumulates intensity values weighted by the narrow and wide Gaussian kernels, with dynamic normalisation to preserve relative magnitude.

Once this is complete, the edge is computed:

```
// Final DoG value  
float edgeDifference = (blurredSum.x - tau * blurredSum.y);
```

Where blurredSum.x and blurredSum.y are the weights for the two Gaussians used, and tau is the threshold adjustment. This gives a successful output however, it is hard to see, so a float value is used to boost the contrast.

```
// Add contrast boost  
edgeDifference *= 100.0;
```

Additionally, the DoG isolates the Y channel when sampling the previous render texture of the bilateral filter passed through. This allows the DoG to only work on the brightness value rather than the chrominance values that would change colour values. This means that the DoG pass has no influence on the overall RGB output at the end of the pipeline and only enhances the stylised contours found in the bilateral filter.

This creates a safe way of implementing further edge detection without messing with the colour values that are important in watercolour rendering.

The final output can be seen in Figure 8: Difference of Gaussians

3.2.1.7 Flow-Guided Difference of Gaussians

A flow-guided Difference of Gaussians is introduced to complete the edge detection that will act as the soft edges for the watercolour post-process. This is because the standard DoG captures edge information but does so uniformly, which can fall short when trying to imitate watercolour. This pass takes in the flow map and the previous DoG textures. The difference of the flow-aligned Difference of Gaussians to the regular DoG is that its samples along the flow map and traces edges following the curvature that are defined within the flow map, rather than stepping uniformly in a fixed direction.

To do this, the tangent vector from the flow map is taken and used to construct the path that updates at every step to follow the curvatures defined from the underlying image. This allows the filter to align with the structure of the image below, producing better edge flow around the image, that better mimic hand-painted contours.

A step size is computed to maintain this consistent sampling along the flow map. This ensures each step corresponds to a movement of one texel in image space. Without this, step spacing would become uneven, especially along steep or shallow curves, leading to jittery, aliased results. By recalculating the step size based on the flow map, the algorithm preserves uniform sampling even when following highly curved flow lines.

This combo of flow traversal and consistency makes edges flow with the image structure rather than uniformly across the whole image, allowing the output to have more expressive contour lines that are used imitate expressive watercolour qualities. Compared to the standard DoG this versions produces lines that feel less mechanical and more natural.

Before finally outputting the value to a render texture, the output is inverted, going from harsh white lines against black, to black outlines against a white background. The project also attempted to apply

posterization in this section however, a few issues occurred with this. This was not possible to obtain due to the outlines being absolute values, being either 1 or 0. Making it impossible to have a smoothstep with values close to those numbers. The output can be seen here in, Figure 9: Flow-Aligned Difference of Gaussians.

3.2.1.8 Bilateral Filtering for Abstraction

After completing the edge detection for the pipeline. The project continues following Kyprianidis and Döllner's (2009) method in where further bilateral filters discussed earlier in section 3.2.1.5 are used on the image. The shader logic and code structure remain identical, including the directional sampling along gradient and tangent vectors from the flow map, its purpose at this stage is different. Instead of being used for edge detection setup, these bilateral filters are focusing on general abstraction of the image to push for further stylisation.

Traditional watercolour paintings, visual detail is often simplified or "softened", especially in regions that don't have sharp contours. This filtering stage replicates that quality by softening residual details, while maintaining the edge structure of the image created that is used in the edge detection passes and flow map. The result is an output where fine textures and a small-scale contrast are suppressed, producing broad regions of cohesive colour that try to imitate the pigment washes on paper.

The bilateral pass occurs 4 times, calling for both directions via a loop within the render function. The variable used for how many times these passes to happen is exposed to the GUI, allowing the user to change how many times the abstraction occurs.

This does introduce a challenge, however, when using too low detailed textures, the bilateral filter can often soften the texture region to where you cannot see the effect of the post-processing effect, this can be found in Figure 14: Importance of Texture Detail.

3.2.1.9 Colour Quantisation, Blending and Paper texture

To finish off the project and the graphics pipeline for the post-processing a few things needed to be done, colour quantisation, blending of the edges and the final colour output, and adding a final paper texture over the screen.

Colour quantisation is done for further stylisation. This stage reduces the number of Luminance levels(Y') while preserving chrominance (C_b and C_r), resulting in large, coherent regions of tone reminiscent of watercolour washes.

In watercolour painting, pigment often pools in discrete tonal bands rather than transitioning smoothly – a quality that the project tries to replicate through colour quantisation. By flattening gradients into distinct tone steps, this effect mimics how colours dry on textured paper, capturing a core trait of watercolour.

In the project implementation, quantisation is applied in the YCbCr colour space, where luminance (Y') is quantised and chrominance (C_b and C_r) are left unaltered to preserve hue fidelity. This helps avoid unnatural colour shifts during the quantisation and maintains a correct palette.

The quantisation is defined as the following:

```
// Hard quantization step
float qY = floor(y * quantLevel + 0.5) / quantLevel;
```

quantLevel being the number of discrete levels, this is set at a value of 8. However, to soften the transitions between these levels and avoid abrupt banding, a smoothstep is used:

```
// Smoothstep softening bands
float smoothY = smoothstep(-2.0, 2.0, transitionSmoothing * (y - qY) * 100.0) - 0.5;
```

This creates a gentle blend around the quantisation threshold, giving edges a more organic transition and improving temporal coherence when the camera moves.

The final quantisation luminance is then calculated:

```
float quantY = qY + smoothY / quantLevel;
```

This subtle softening tries to simulate how watercolour pigment transitions are not perfectly uniform, particularly in areas where colours

merge due to water flow or uneven drying. However, the outcome is still too uniform and does not resemble watercolour pigmentation correctly but has a solid foundation to build upon, Figure 10: CQ - Colour Quantisation.

Next, the second last pass is a blending stage, which combines the colour quantisation pass with the flow-guided DoG filter. This pass integrates structural detail into the final image without introducing heavy contrast or losing the stylistic softness characteristics of watercolour.

Many blending techniques were tried to get the correct output. However, the project continues to follow the Kyprianidis and Döllner (2009) method. Instead of directly overlaying black ink-like strokes, the blending is performed only on the luminance (Y') channel of the quantised YCbCr colour. This ensures that hue and chromatic fidelity are preserved, and that edge information only affects tonal contrast, not colour.

The blending technique that had the best outcome modulates the quantised luminance using the strength of the DoG edge map to avoid harsh attenuation. The edge values are remapped using a lerp function:

```
edge = lerp(0.5, 1.0, edge);
```

This gently boosts weaker edge values while keeping strong edges from fully blacking out the luminance. The luminance is then scaled and clamped:

```
float y = saturate(edge * ycbcr.r);  
y = max(y, 0.1); // ensure Y doesn't collapse
```

This avoids unnatural dark regions and maintains the soft, diffuse qualities of watercolour, where strokes bleed and flow into one another rather than forming harsh outlines.

By applying edge detail through multiplicative luminance scaling, the blending creates a subtle dry-brush look, reinforcing the paper and pigment interaction in real-world watercolour. It also helps to visually anchor tonal transitions that were softened in the quantisation stage, improving legibility without sacrificing stylisation.

Alternative edge combination methods (such as screen or additive blending) were explored during development but often produced either

overly harsh contrasts or flattened the stylised tone layers. The final approach strikes a balance between preserving edge integrity and maintaining soft tonal harmonies, a key aesthetic goal for NPR watercolour simulation. Figure 11: Combining CQ and Edge Abstraction.

To complete the stylisation part of the pipeline before reverting into the RGB colour space for final output, a final paper texture is applied over the rendered scene. This texture adds a layer of physicality and grounding, reinforcing the traditional watercolour aesthetic by adding a feeling of hand-painting on a canvas.

In early development the paper texture was simply blended over the as a full screen overlay, However, this approach failed to preserve depth, causing the texture to appear stuck to the screen rather than integrated into the 3D environment. As a result, the final image appeared overly flat and artificial, diminishing the illusion of painted surfaces receding into space.

To address this, depth-aware blending was introduced. A sampled linearised depth texture is used to modulate the blending strength, allowing closer objects to receive more pronounced paper detail while distant areas fade the texture out, this creates a more convincing illusion that stylised strokes are interacting with a canvas in physical space.

In the implementation, both the final rendered colour, paper texture and depth texture are sampled. The depthValue (being the depth texture sample) is non-linearly remapped to concentrate influence in a mid-range, improving blending transitions:

```
depthValue = pow(depthValue, 2.2);  
depthValue = smoothstep(0.2, 0.8, depthValue);
```

A dynamic blend is then calculated:

```
float dynamicBlend = saturate(blendStrength + (depthValue * depthFactor));
```

The output is computed using an overlay blend, which enhances contrast in dark areas and softens it in lighter ones:

```
float3 blendOverlay = (baseColor < 0.5  
    ? (2.0 * baseColor * paperColor)  
    : (1.0 - 2.0 * (1.0 - baseColor) * (1.0 - paperColor)));
```

Finally, the blend is mixed with the original colour using lerp which then creates the final output.

This process ensures the paper texture does impersonate the feeling of being flat but rather 3D, Figure 12: Paper Texture Overlay.

3.2.2.0 Temporal Blending

Throughout the process, temporal incoherence has been detected, especially when camera movement is active and when pixel values are highly different, creating flickers in the scene, but also when the ocean is moving, little contour jitters can occur. To try combat this, temporal blending was implemented in the pipeline. This pass blends the current frame's stylised output with a reprojected version of the previous frame, reducing abrupt visual changes and preserving the continuity of stylised features like brushstroke edges.

The temporal blending is achieved by linearly interpolating between the current frame and the previous frame using a weighting factor:

```
return currentTex * blendStrength + previousTex * prevWeight;
```

This approach is inspired by (Redorav, 2022). In this implementation, the weighting factor is set between 0.1 and 0.2 to balance responsiveness and stability, however, this project lets the user determine this value through the GUI.

This temporal technique, however, did not solve the issues of the flickering in areas where they were present beforehand, for the future being able to pass multiple previous frames instead of just 1 may help this factor. Another main point of future work would be to implement motion vector-based maps.

3.2.2.1 Colour Conversion and Comparison Shader

After all stylization and post-processing operations are complete, the image exists in the Y'CbCr colour space, where luminance (Y') and chrominance (Cb, Cr) were separated to enable perceptually accurate edge detection, bilateral filtering, and quantization. Before displaying the final image, the data must be converted back into the standard RGB colour space.

The conversion is performed using an inverse transformation of the matrix used earlier in the pipeline to go from RGB to YCbCr. The standard inverse YCbCr to RGB conversion is applied as follows:
(equation)

This transformation restores the RGB colour space, also before this multiplication the chrominance values (Cb and Cr) are explicitly re-centred around 0 by subtracting a constant offset of 0.5. This step ensures the input aligns correctly with the matrix's expected value range and prevents colour distortion.

Now that all shaders for the effect have complete, a user tool to verify the effects of the stylization but also to see each pass and what they do was created. The comparison pass allows toggling between the input (pre-processed scene) and output (post-processed scene) frames. A simple full screen blend shader is used to display the original image and the render texture of any pass as the user chooses, for example this pass allows the user to look at the flow map compared to the original untouched scene. This has proven to be especially helpful throughout development and testing, as it enables rapid validation of how effectively individual pipeline components contribute to the overall stylization.

Chapter 4 Results

This chapter presents the results of the implemented post-processing pipeline designed to simulate watercolour effects. The aim is to evaluate how successfully the technique replicates the stylistic elements of watercolour, maintains frame-to-frame temporal coherence, and performs under real-time constraints in a 3D environment.

Dividing the results into four sections, first the visual quality output is examined against the intended aesthetic and other attempts of recreation. This is followed by an analysis of frame stability to assess temporal coherence. Lastly, GPU timings for each render pass are evaluated to determine real-time viability.

The system used to retrieve the data is as follows:

CPU – AMD Ryzen 5 7600 6-Core Processor

GPU – NVIDIA GeForce RTX 4060

RAM – 32GB DDR5

OS – Windows 11

4.1 Stylisation output - Simulating watercolour effects

The primary objective of this project was to replicate watercolour through a real-time, image-space post-processing pipeline. Key features targeted include pigment diffusion, soft tonal transitions and expressive contouring.

To evaluate the effectiveness of the stylisation, the output was analysed using the qualitative matrix defined earlier in *Section 3.1.5*. The same image that has been used previously to show results within (Curtis et al. (1997) and Kaplan et al. (2006) was passed into the project's pipeline and will be evaluated according to the matrix criteria.

Refer to Figure 13: Comparison with Curtis and Bousseau result: (a) original image, (b) Curtis watercolour result, (c) Bousseau watercolour result, (d) my result. The images show the different NPR technique outputs being used to evaluate the pipeline created.

Trait	(b) Curtis	(c) Bousseau	(d) Ours
Edge softness	Sharp	Sharp	Soft
Tonal Transitions	Subtle	Moderate	Very
Flow Alignment	Partial	Good	Partial
Paper Influence	Strong	Mild	Mild
Pigment Diffusion	Smooth	Organic	Hard edges
Edge abstraction	Stylized	Slightly Stylized	Stylized

Table 2: Evaluated NPR Techniques, using Evaluation Qualitative Matrix

The ratings shown in *Table 2: Evaluated NPR Techniques, using Evaluation Qualitative Matrix*, reflect how well each method captures the key traits defined in the qualitative matrix. Our implementation consistently produced soft edges, and stylized contours, which aligns with the desired watercolour aesthetic. However, areas like flow alignment, tonal transitions, paper influence, and pigment diffusion highlights refinement is needed. A detailed discussion of how and why these values were graded is provided in Section 5.1.1.

Figure 14: Importance of Texture Detail demonstrates the importance of high-frequency texture detail in producing effective results for the post-processing effect. Both images were captured from the same angle and perspective to ensure a fair comparison. The left image, which uses low-detailed textures, results in less expressive diffusion and weaker edge abstraction. In contrast, the right-hand image that uses a more detailed surface texture produces ricker pigment variation and more visually distinctive stylised contours. This highlights how surface detail contributes to the overall aesthetic outcome of the pipeline.

4.2 Temporal Coherence and Frame Stability

Refer to Figure 15: Stylisation inconsistencies across frames for this section.

To evaluate temporal coherence, three frames were captured during object movement. The camera parameters, position and vertical angle were constant across all captures to identify the inconsistencies. Additionally, a low edge smoothing value was used during this test to make abstraction more visible. As shown in Figure 15, the stylisation effect, particularly the edge abstraction, lacks consistency over time. Dark outlines along the barrels and railing appear, disappear, or shift between frames. While this abstraction is hard to see in still images, this frame-to-frame inconsistency becomes more visible in motion, reducing the visual stability of the output. This temporal instability disrupts the visual quality and experience for the user.

Additionally, the temporal blending pass, implemented to mitigate these inconsistencies, produced visible ghosting instead. As shown in Figure 16, remnants of the previous frame persist as a faint silhouette. Rather than smoothing over inconsistencies, this method introduces new artefacts, reducing clarity and amplifying motion blur.

4.3 Performance and Real-Time Viability

Render pass	GPU Time (ms)
RGB to YCbCr	0.04
Structure Tensor (ST)	0.14
ST Horizontal Blur	0.09
ST Vertical Blur and Flow Map	0.09
Bilateral Filter	1.21 (4 passes total, 0.07 per pass)
DoG	0.05
Flow-based DoG	0.15
Colour Quantisation	0.04
Combine Shader	0.05
YCbCr to RGB	0.04
Paper Overlay	0.08
Additional Passes	GPU Time (ms)
Temporal Blend	0.12
Comparison Shader	0.11

Table 3: Render Pass GPU Times (ms)

The total GPU time without additional passes is 1.98ms and 2.21ms with. This performance supports real-time rendering well under typical 60fps limits.

To view the pie chart of the table information, refer to Figure 16: GPU Usage of each Render Pass

The pipeline was profiled using GPU timings per render pass to assess if the pipeline is feasible in real-time. As visualised in Figure 16: GPU Usage of each Render Pass, the bilateral filter accounts for the majority of GPU time, consuming around 61% of the rendering cost due to multiple passes required for abstraction smoothing. Other major contributors include the Flow-based DoG and Structure Tensor, reflecting the complexity of techniques that extract features. Despite this, the

average total render time remains within viable limits for real-time post-processing applications.

Chapter 5 Discussion

This chapter critically evaluates the effectiveness of the watercolour stylisation pipeline, both technically and aesthetically. It reflects on the results gathered from the qualitative testing and quantitative performance profiling and discusses how well the project meets its aims.

5.1 Results Discussion

5.1.1 Visual Fidelity and Stylisation Effectiveness

The evaluation of stylisation quality was guided by the qualitative matrix defined in Section 3.1.5, which outlined six key traits that are associated with traditional watercolour: edge softness, tonal transitions, flow alignment, paper influence, pigment diffusion and edge abstraction. Each of these traits was rated to stylised outputs from existing NPR methods, Curtis et al. (1997) and Kaplan et al. (2006) and the output of this project, as summarised in Table 2: Evaluated NPR Techniques, using Evaluation Qualitative Matrix and illustrated in Figure 13: Comparison with Curtis and Bousseau result: (a) original image, (b) Curtis watercolour result, (c) Bousseau watercolour result, (d) my result.

In the context of the matrix, edge softness was defined as the extent to which boundaries between colour regions dissolve naturally, mimicking the way pigment bleeds into surrounding water on the paper. A strong result would show blurred, diffuse contours rather than crisp edges. In the project's output (Figure 13d), edge softness was achieved to a moderate extent through the flow-aligned Difference of Gaussians, especially around the upper curves of the fruit. While not as sharp as Curtis' output (Figure 13b), the edges in this implementation lacked the full fluidity of traditional watercolour, appearing selectively softened rather than uniformly diffuse.

Tonal transitions were judged based on their ability to replicate the gentle, layered blending typical of watercolour washes. A strong rating would involve gradual transitions between light and dark values with a sense of pigment layering and fade. In contrast, the output produced more abrupt tonal transitions due to aggressive colour quantisation. Although this enhanced stylisation gave the image a painterly look, it created a posterised effect with minimal smooth stepping. Compared to Boisseau's rendering (Figure 13c), where tonal transitioning is smooth and continuous throughout the image, but especially on the green apple.

Flow alignment refers to the directional consistency of the stylisation effects with the underlying geometry or brush stroke direction. A high-quality result would show features like contours, diffusion or shading following the natural curvature of objects. In this project, flow was derived from a smoothed structure tensor that was used to create a flow map, which guided stylisation in the flow-aligned difference of Gaussians. While some flow alignment is visible along object edges in Figure 13d, other areas, particularly flat or low detail textures, show little or no coherent direction. This reduced the effectiveness of flow-aligned stylisation, making some areas feel visually disconnected from the object's geometry.

Paper influence was defined in the matrix as the visible impact of paper grain or texture on the appearance of pigment accumulation. A strong implementation would show how texture affects colour in a way that mimics absorption in real paintings. In this project, a canvas texture was overlaid in a render pass, with intensity controlling strength and how it's affected. However, the result remained minimal as it only appeared as a texture and did not have an impact on colour. Compared to Curtis's output (Figure 13b), where paper grain visibly affects colour.

Pigment diffusion, a central quality of watercolour, involved soft, irregular spreading of colour within regions, mimicking the flow of pigment through

wet surfaces. Strong diffusion effects feature soft gradients, unpredictable blooming, and edge irregularity. In this implementation, diffusion was not achieved. Despite the use of bilateral filters, the result showed hard transitions and lacked the fluidity seen in real watercolour. As seen in Figure 13d, pigment appears too cleanly separated and consistent, with transitions that are more digital than organic.

Edge abstraction was defined as the stylised exaggeration or simplification of edges to mimic brush strokes. A good implementation emphasises expressive, varied linework that simplifies without fully discarding structure. This project performed well in this area, combining Difference of Gaussians with a flow-guided Difference of Gaussians to generate bold, stylised outlines. These lines adapt directionally based on flow field inputs and visually resemble hand-drawn brush strokes. Figure 13d shows clean and expressive edge work that contributes significantly to the overall stylisation.

In summary, applying the evaluation matrix revealed that the pipeline's strengths lie in expressive contouring and stylised edge abstraction, supported by the Dog and FDoG filtering. Moderate success was achieved in edge softness through bilateral filtering. These outcomes reflect partial fulfilment of the project's aim: to simulate natural colour bleeding, pigment diffusion and soft edge behaviour observed in traditional watercolour paintings. While the pipeline effectively establishes a stylised aesthetic, the lack of fluid pigment behaviour and organic blending results resembles acrylic painting more than a watercolour effect. This work forms a solid foundation for extending these render passes to simulate more watercolour traits.

Beyond the specific traits in the matrix, texture quality also played a significant role in the visual effectiveness of the post-process. As illustrated in Figure 14: Importance of Texture Detail, models with high-frequency surface detail (right) created stronger edge abstraction and

more varied pigment behaviour compared to the low-detail textures (left), which resulted in flatter, less expressive outputs. This is primarily due to the reliance on the structure tensor for generating the flow map, which guides flow-aligned stylisation such as the FDoG filter. When textures lack sufficient change/detail, the tensors' eigenvector estimation becomes unstable, producing near-zero magnitudes. In such cases, the program defaults to a fallback flow direction to avoid division by zero, as seen in the flow map shader. This fallback removes meaningful directional cues, leading to inconsistent or directionless stylisation. As a result, features like directional abstraction and pigment alignment become less coherent, reducing the watercolour effect. This highlights the importance of detailed texture usage when using this post-process.

5.1.2 Temporal Coherence and Frame Stability

The results presented in Section 4.2 (Figure 15: Stylisation inconsistencies across frames) demonstrated clear issues with temporal instability across frames, particularly in the outlines. Despite fixed camera settings, contours along the barrels and railings shifted or disappeared unpredictably, reducing the visual consistency of the effect during motion. This instability is a known challenge in stylised post-processing, where high-frequency edge abstraction can be sensitive to small pixel changes. In this project, bilateral filtering, particularly in the abstraction phase, helped suppress some of this flicker by smoothing image detail and reducing noise sensitivity between frames. Without this step, artefacts caused by per-frame changes in the flow map and edge detection would have been more pronounced.

To further improve consistency, the temporal blending pass was introduced to combine the previous frame with the current one. While this reduced low-motion regions, it introduced ghosting artefacts instead, which are residual silhouettes from the previous frame. As shown in Figure 15: Stylisation inconsistencies across frames, these silhouettes blurred moving edges and reduced the visual clarity of the outlines, making the effect feel smeared rather than coherent.

Ultimately, although the pipeline tried to address temporal coherence, the results show that the pipeline does not fully meet the project's aim of maximising visual consistency in stylised real-time rendering. While bilateral filtering offered improvement, the temporal blending strategy ultimately was counterproductive. Future implementations would benefit from methods such as motion vector maps to achieve stable, painterly results across frames.

5.1.3 Performance and Real-Time Viability

One of the project's aims was to maintain real-time performance, ensuring the stylisation pipeline could run smoothly within a typical 60fps rendering budget. The results shown in Section 4.3 confirm that this goal is largely met, with the pipeline achieving GPU times between 1.98ms and 2.21ms, well below the 16.6ms per-frame budget.

According to the results *in* Figure 16: GPU Usage of each Render Pass, the stage that is the most computationally expensive is the Bilateral filter stage of the post-process. This is due to its repetition in the pipeline, 4 passes in total are used before and after edge smoothing. This repetition is not fixed, however, meaning through the project's GUI, the user can choose how many times the pipeline will pass through the bilateral filter, depending on the desired balance between performance and outcome quality.

This control is vital as the bilateral filter plays a crucial role in suppressing visual artefacts such as flickering and jittering, as it smooths out surfaces. While it contributes heavily to the computational overhead of the pipeline, it is also one of the more visually and functionally impactful passes in the pipeline. Reducing its usage would improve rendering time, but it would also compromise the soft edge behaviour and abstraction smoothness that are integral for simulating watercolour paintings.

This trade-off shows NPR visual quality with computational overhead, meaning careful software design must balance artistic value with runtime restraints. While the pipeline showed no major performance bottlenecks in testing, further optimisations could still be implemented if the pipeline was further developed, for example, reusing render textures when values remain unchanged. This would reduce memory use and improve throughput, making the pipeline more viable for real-time game applications, especially if the project was scaled.

5.2 Overall Results Discussion

This chapter has critically examined the strengths and limitations of the pipeline, drawing from both qualitative and quantitative data. While the project delivered a visually distinct, stylised result with real-time performance, it fell short in replicating key watercolour characteristics such as fluid pigment diffusion and temporal coherence around edges within high-frequency areas.

Chapter 6 Conclusion and Future Work

This project set out to develop a real-time, image-space post-processing pipeline capable of replicating traditional watercolour aesthetics, with a focus on simulating natural pigment diffusion, soft edge behaviour, and how paper texture affects the fluidity of the paint and achieving frame-to-frame temporal coherence. Alongside these artistic goals, it aimed to maintain computational efficiency suitable for a 3D game environment, with stylisation performance staying within real-time rendering budgets. The implementation, based on Kyprianidis and Döllner (2009) that uses bilateral filtering, structure tensors, and flow-aligned Difference of Gaussians, was used to achieve a recognisable result that captured some but not all the desired traits. The strongest outcomes were observed in edge abstraction and expressive contour rendering, where the combined DoG and FDoG contributed to this fact. The system also met real-time goals, with rendering times comfortably within a 60fps budget.

However, the results also exposed several areas where the pipeline fell short of the project's aims, to replicate natural watercolour behaviours. Key traits such as pigment diffusion, flow-aligned brushwork, and subtle tonal blending were either weakly represented or missing altogether. The effect leaned more toward an acrylic-like look, lacking the organic fluidity of watercolour traits. Temporal coherence also remained a challenge, with per-frame stylisation changes causing visible instability around edges that were in high-frequency areas, and the temporal blending pass was ultimately counterintuitive.

Knowing these issues with the current pipeline, future improvements can be made to create a better natural watercolour output whilst being temporally coherent. These improvements are as follows:

- Flow Guided Colour Bleeding:
 - The existing flow map could be adapted to control the direction of colours and how they would bleed into areas around it, allowing simulation of wet-on-wet effects or dry-on-dry effects.
- Better Paper Integration:
 - Allowing pigment to respond dynamically to paper grain, rather than overlaying a static texture, which would increase realism, especially in light or blended areas.
- Advanced Temporal Coherence:
 - Replacing the current frame blending approach with motion vector-based mapping would help stabilise stylisation across moving objects without introducing ghosting.
- Shader Optimisation:
 - Introducing logic to reuse intermediate render textures when values remain unchanged could reduce memory use and further improve performance.

In summary, this project is a mixed success. While it met some of the goals, such as performance and had a painterly style, it fell short of fully replicating watercolour traits such as pigment diffusion and temporal consistency around edges. Nonetheless, the project creates a strong foundation for future development, as it offers a modular system that can be used to refine the post-process into a more expressive and authentic watercolour post-processing effect.

List of References

- Pope, L. (2018). *Return of the Obra Dinn* [video game]. 3909 LLC.
- Clover Studio. (2006). *Ōkami* [video game]. Capcom.
- Curtis, C.J., Anderson, S.E., Seims, J.E., Fleischer, K.W. and Salesin, D.H. (1997) 'Computer-generated watercolor', *Proceedings of the ACM SIGGRAPH*, pp. 421–430. Available at: <https://dl.acm.org/doi/pdf/10.1145/258734.258896> (Accessed: 17 March 2025).
- Bousseau, A., Kaplan, M., Thollot, J. and Sillion, F.X. (2006) 'Interactive watercolor rendering with temporal coherence and abstraction', *Proceedings of the 4th International Symposium on Non-Photorealistic Animation and Rendering (NPAR '06)*, pp. 141–149. Available at: <https://doi.org/10.1145/1124728.1124751> (Accessed: 17 March 2025).
- Winnemöller, H., Kyprianidis, J.E. and Olsen, S.C. (2011) 'XDoG: An eXtended difference-of-Gaussians compendium including advanced image stylization', *Computers & Graphics*, 35(6), pp. 1309–1316. Available at: <https://doi.org/10.1016/j.cag.2011.06.002> (Accessed: 8 April 2025).
- Marr, D. and Hildreth, E. (1980) 'Theory of edge detection', *Proceedings of the Royal Society B: Biological Sciences*, 207(1167), pp. 187–217. Available at: <https://royalsocietypublishing.org/doi/10.1098/rspb.1980.0020> (Accessed: 6 March 2025).
- López, E. (2022) 'Temporal AA and the quest for the holy trail', *The Code Corsair*, 2 January. Available at: <https://www.elopezr.com/temporal-aa-and-the-quest-for-the-holy-trail/> (Accessed: 15 January 2025).
- Bénard, P., Bousseau, A. and Thollot, J. (2011) 'State-of-the-Art Report on Temporal Coherence for Stylized Animations', *Computer Graphics Forum*, 30(8), pp. 2367–2386. Available at:

- <https://doi.org/10.1111/j.1467-8659.2011.02075.x>. (Accessed: 5 October 2024).
- Vanderhaeghe, D. et al. (2007) Dynamic point distribution for stroke-based rendering. Available at: <https://doi.org/10.2312/EGWR/EGSR07/139-146>. (Accessed: 5 October 2024).
 - Möller, T. et al. (2018) Real-time rendering. Boca Raton: Crc Press, Taylor & Francis Group.
 - Kyprianidis, J.E., Kang, H. and Döllner, J. (2009) 'Image and Video Abstraction by Anisotropic Kuwahara Filtering', *Computer Graphics Forum*, 28(7), pp. 1955–1963. Available at: <https://doi.org/10.1111/j.1467-8659.2009.01574.x>. (Accessed: 5 October 2024)
 - Kumar, M.P.P., Poornima, B., Nagendraswamy, H.S. and Manjunath, C. (2019) 'A comprehensive survey on non-photorealistic rendering and benchmark developments for image abstraction and stylization', *Iran Journal of Computer Science*, 2, pp. 131–165. Available at: <https://doi.org/10.1007/s42044-019-00034-1> (Accessed: 28 April 2025).
 - Ciao, S., Guan, Z., Liu, Q., Wei, L.-Y. and Wang, Z. (2024) 'Ciallo: GPU-accelerated rendering of vector brush strokes', *Special Interest Group on Computer Graphics and Interactive Techniques Conference Papers*, pp. 1–11. Available at: <https://doi.org/10.1145/3641519.3657418> (Accessed: 7 November 2024).
 - Kyprianidis, J.E. and Döllner, J. (2009) 'Real-time image abstraction by directed filtering', in Engel, W. (ed.) *ShaderX7: Advanced Rendering Techniques*. Boston, MA: Charles River Media, pp. 285–295.

Bibliography

- Acerola (2023) *This is the Difference of Gaussians*. Available at: https://www.youtube.com/watch?v=5EuYKEvugLU&ab_channel=Acerola (Accessed: 8 November 2024)
- Acerola (2023) *This is the Kuwahara Filter*. Available at: https://www.youtube.com/watch?v=LDhN-JK3U9g&ab_channel=Acerola (Accessed: 8 November 2024)
- Barney Codes (2024) *Introduction to shaders: Learn the Basics!*. Available at: https://www.youtube.com/watch?v=3mfvZ-mdtZQ&ab_channel=BarneyCodes (Accessed: 20 October 2024)
- Freya Holmér (2021) *Shader basics, Blending and Textures – Shaders for Game Devs [Part 1]*. Available at: https://www.youtube.com/watch?v=kfM-yu0iQBk&ab_channel=FreyaHolm%C3%A9r (Accessed: 20 October 2024)
- Lalaoopybee (2023) *difference of gaussians(dog)*. Available at: <https://www.shadertoy.com/view/msySR1> (Accessed: 9 November 2024)

Appendices

Appendix A – Images

A.1 – Original Scene



Figure 1: Original Scene, including water, ship model and moving objects

A.2 – Colour Conversion RGB – YCBCR

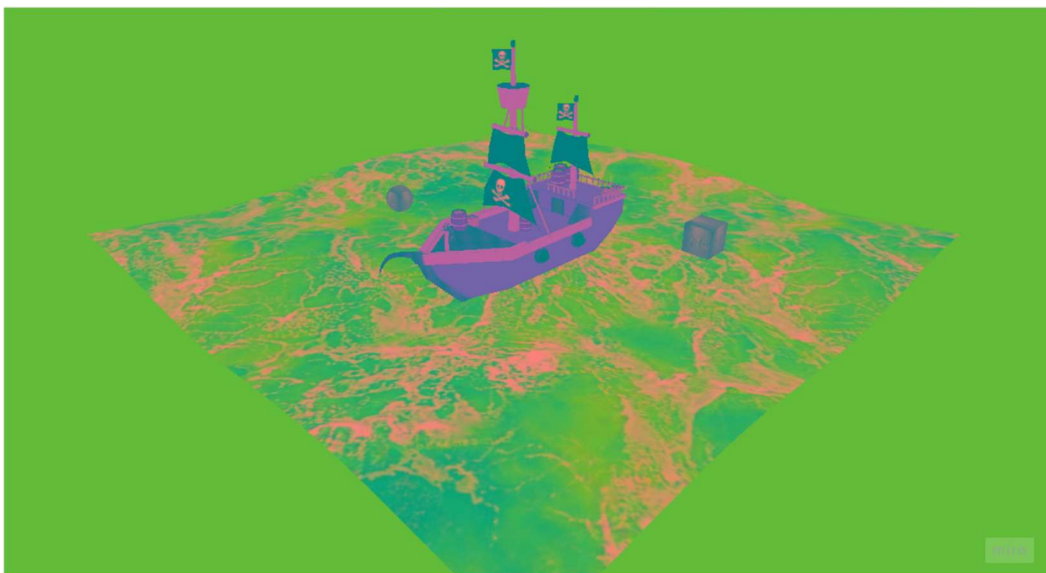


Figure 2: RGB to YCBCR conversion

A.3 – Structure Tensor

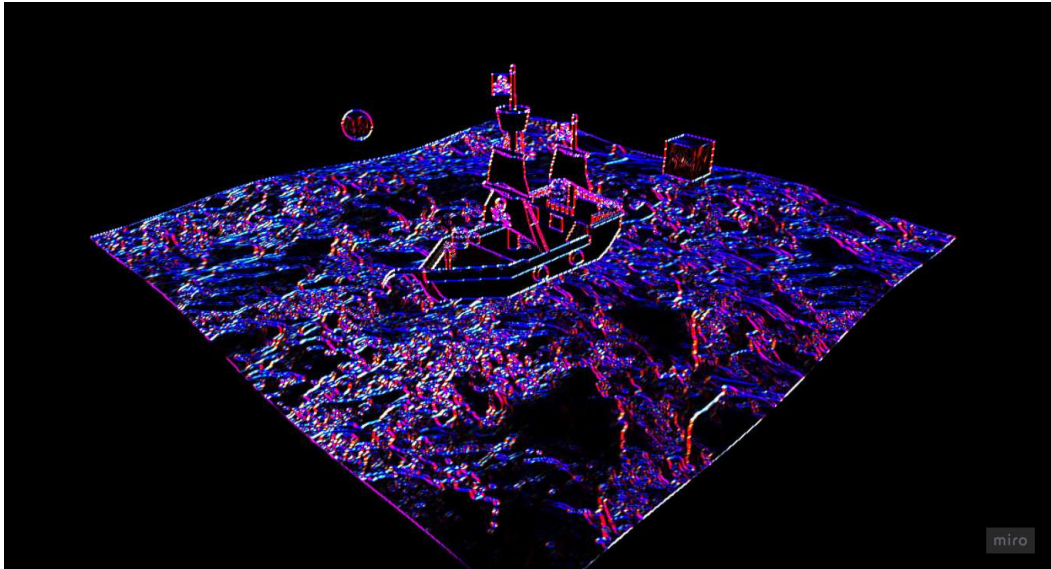


Figure 3: Structure Tensor

A.4 - Structure Tensor – Horizontally Smoothed

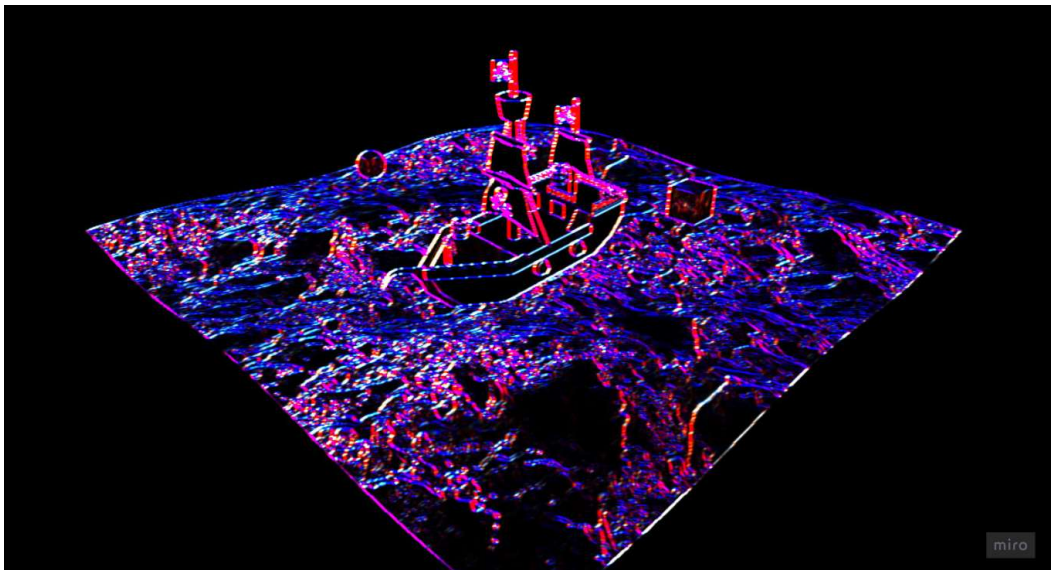


Figure 4: Horizontally smoothed Structure Tensor

A.5 – Flow Map Generation

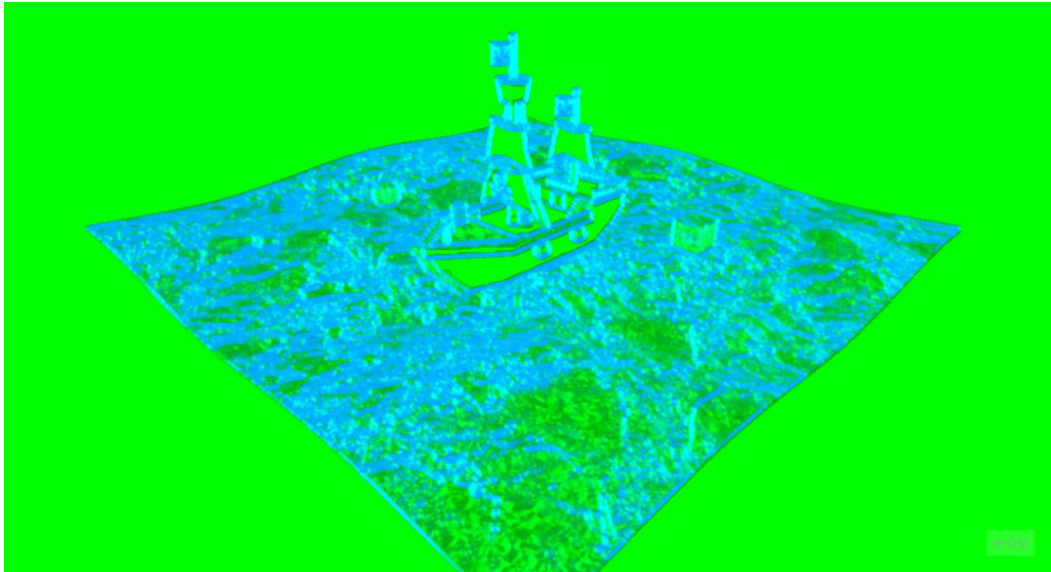


Figure 5: Vertical smoothed Structure Tensor and Flow map Generation

A.6 – Bilateral Filter - Vertical



Figure 6: Vertical Bilateral Filter

A.7 – Bilateral Filter – Horizontal



Figure 7: Horizontally smoothed Bilateral Filter

A.8 – Difference of Gaussians

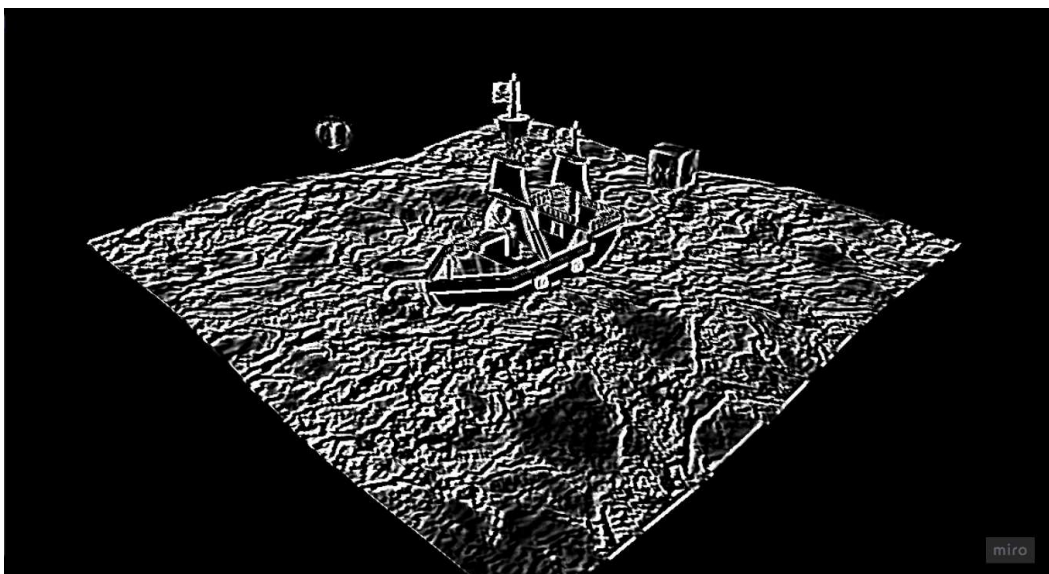


Figure 8: Difference of Gaussians

A.9 – Flow-Guided Difference of Gaussians

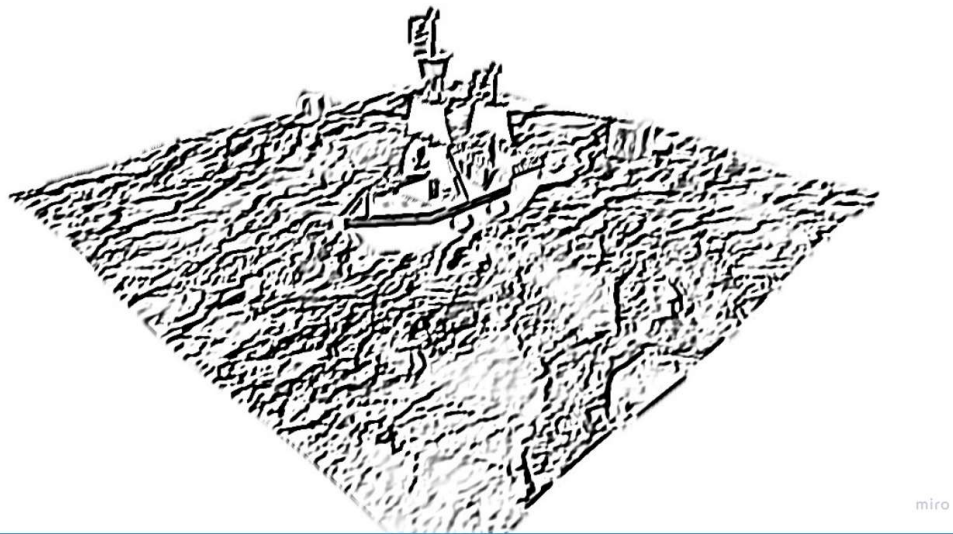


Figure 9: Flow-Aligned Difference of Gaussians

A.10 – Colour Quantisation



Figure 10: CQ - Colour Quantisation

A.11 – Combining Shader



Figure 11: Combining CQ and Edge Abstraction

A.12 – Paper Overlay



Figure 12: Paper Texture Overlay

A.13 – NPR watercolour outputs



Figure 13: Comparison with Curtis and Bousseau result: (a) original image, (b) Curtis watercolour result, (c) Bousseau watercolour result, (d) my result

A.14 – Texture Detail Comparison

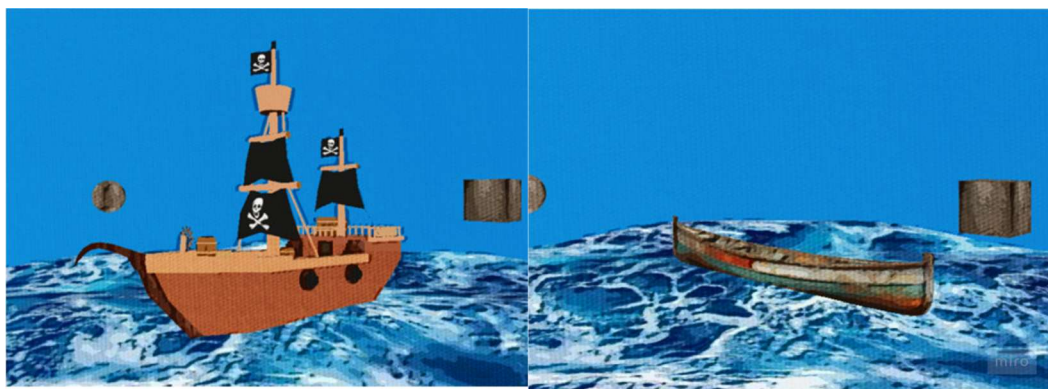


Figure 14: Importance of Texture Detail

A.15 - Temporal Incoherence

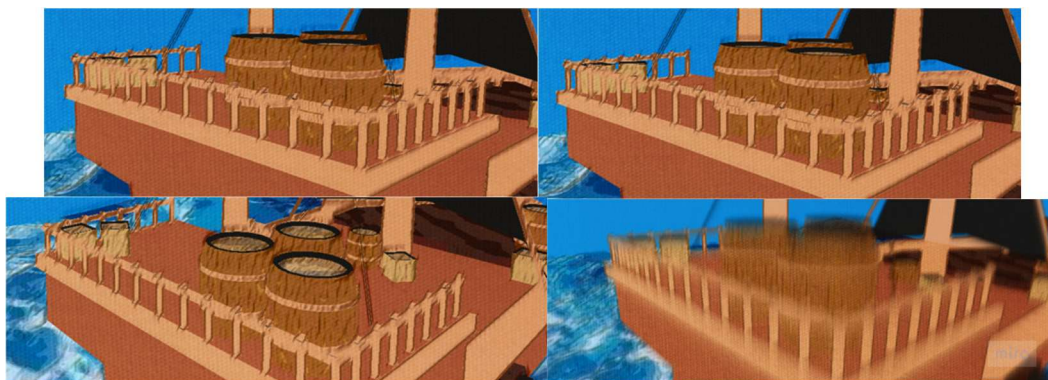


Figure 15: Stylisation inconsistencies across frames

A.16 – GPU Usage

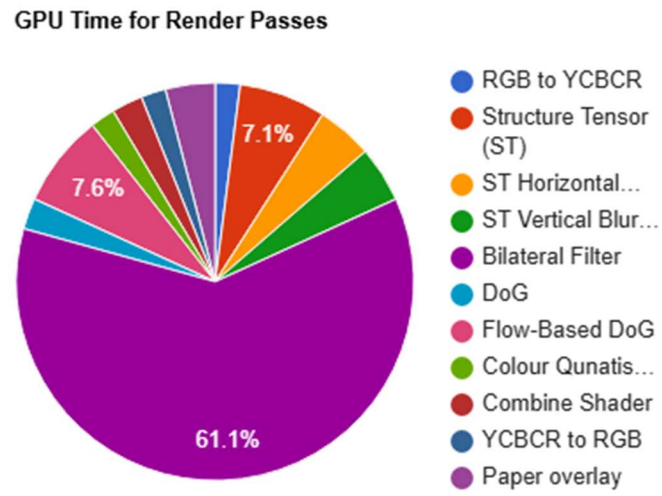


Figure 16: GPU Usage of each Render Pass

A.17 – Project Flow Chart

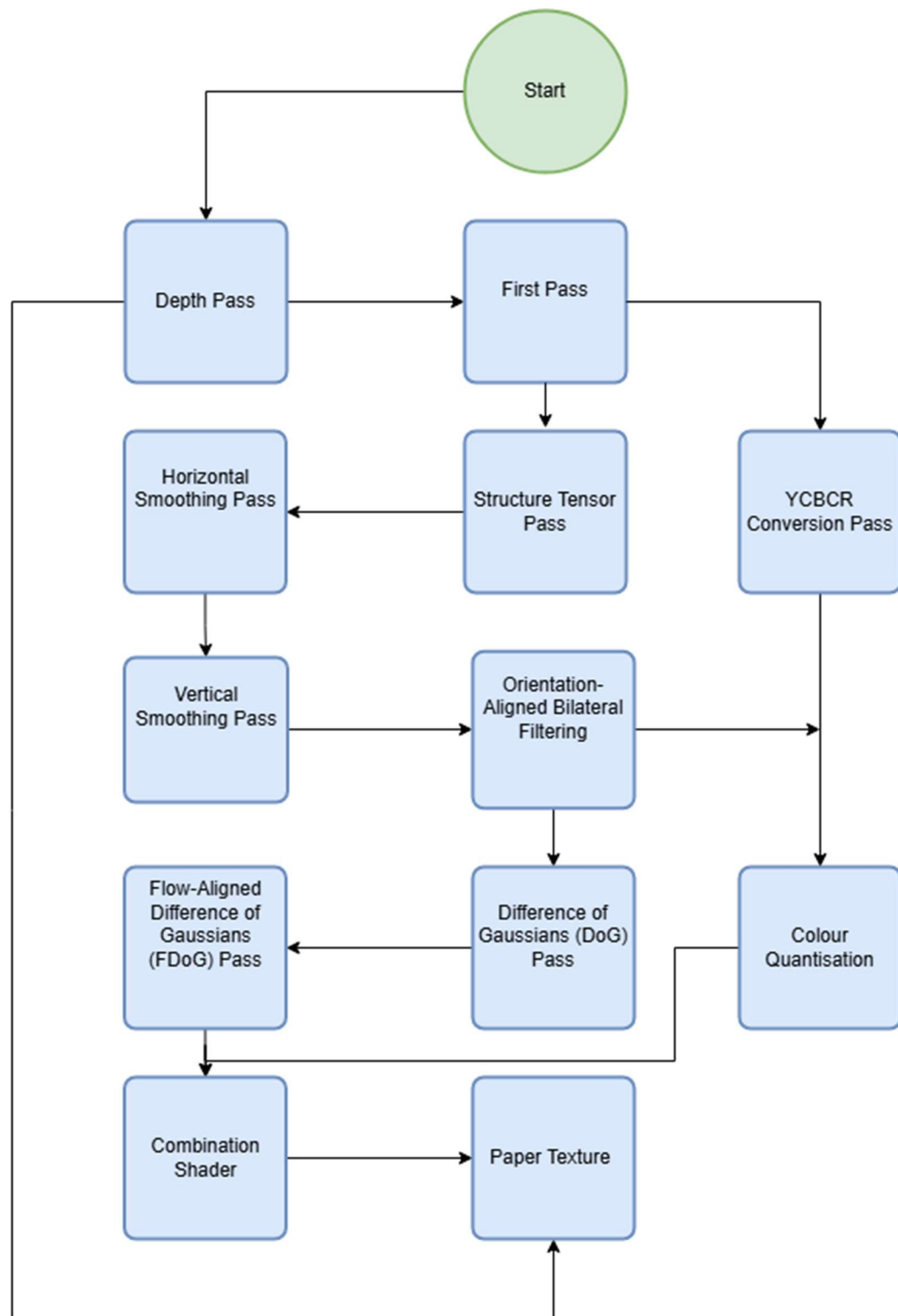


Figure 17: Pipeline Flow chart

A.18 – Data Form


**Abertay
University**

GDPR Research Data Management
Data Sign Off Form

For undergraduate or postgraduate student projects supervised by an Abertay staff member.
This form MUST be included in the student's thesis/dissertation. Note that failure to do this will mean that the student's project cannot be assessed/examined.

Part 1: Supervisors to Complete

By signing this form, you are confirming that you have checked and verified your student's data according to the criteria stated below (e.g., raw data, completed questionnaires, superlab/Eprime output, transcriptions etc.)

Student Name:	Daniel Wooldridge		
Student Number:	2101751		
Lead Supervisor Name:	Erin Hughes		
Lead Supervisor Signature	<i>Erin E Hughes</i>		
Project title:			
Study route:	PhD ☐	MbR ☐	MPhil ☐
	Undergraduate ☒	PhD by Publication ☐	

Part 2: Student to Complete

	Initial here to confirm 'Yes'
I confirm that I have handed over all manual records from my research project (e.g., consent forms, transcripts) to my supervisor for archiving/storage	Yes
I confirm that I have handed over all digital records from my research project (e.g., recordings, data files) to my supervisor for archiving/storage	Yes
I confirm that I no longer hold any digital records from my research project on any device other than the university network and the only data that I may retain is a copy of an anonymised data file(s) from my research	Yes
I understand that, for undergraduate projects, my supervisor may delete manual/digital records of data if there is no foreseeable use for that data (<u>with the exception of consent forms, which should be retained for 10 years</u>)	Yes

Student signature : *DW*

Date: 30/04/2025

